

LangChain

A Practical Reference Guide

Theory, working code, and sample output for the core LangChain building blocks — prompts, chains, agents, memory, vector stores, RAG and LCEL.

*Compiled from the [GeeksforGeeks LangChain Tutorial](https://www.geeksforgeeks.org/data-science/langchain-tutorial/) and its linked articles
([geeksforgeeks.org/data-science/langchain-tutorial](https://www.geeksforgeeks.org/data-science/langchain-tutorial/))*

Prepared for personal study — printer-friendly, black & white layout.

Contents

1. Introduction to LangChain
2. Modules of LangChain
3. Environment & API Setup
4. Prompt Templates
5. ChatPromptTemplate
6. Few-Shot Prompting
7. Output Parsers
8. LLM Chains
9. Agents and Tools
10. Memory in LangChain
11. Vector Stores in LangChain
12. Retrieval-Augmented Generation (RAG) with LangChain
13. LangChain Expression Language (LCEL)
14. Further Topics & Reference Links

1. Introduction to LangChain

LangChain is an open-source framework, available in Python and JavaScript, that makes it easier to build applications on top of large language models (LLMs). Rather than calling an LLM directly and hand-rolling the plumbing around it, LangChain gives you ready-made building blocks for connecting models to external data, tools, and multi-step workflows.

Key Components

- Chains – a sequence of steps where each step can call an LLM, transform data, or invoke a tool. A chain can be a single call or a multi-step pipeline.
- Prompt Management – templates that control the input, output format, and behaviour of a model.
- Agents – LLM-driven components that decide, at runtime, which tool or action to invoke.
- Vector Database – stores data as vectors so similar items can be retrieved by meaning, not just keywords.
- Models – a common interface over many LLM providers (OpenAI, Gemini, Anthropic, Hugging Face, etc.).
- Memory Management – keeps track of context across turns of a conversation or workflow.

How a Typical RAG Pipeline Works

- Documents are split into smaller chunks so they can be processed and embedded efficiently.
- Each chunk is converted into an embedding vector that captures its meaning.
- Embeddings are stored in a vector database, forming a searchable knowledge base.
- A user submits a query, which is itself converted into a vector representation.
- That vector is compared against the stored vectors to find the closest semantic matches.
- The most relevant chunks are pulled from the vector store as context.
- The retrieved context plus the original query are handed to an LLM, which generates the final, grounded answer.

Minimal Working Example

The example below installs LangChain with a Google Gemini integration, sends a single prompt, then builds a small reusable chain using LCEL's pipe (|) syntax: a prompt template feeds the model, and the model's response is cleaned up by an output parser.

Step 1 - Install dependencies

Code:

```
!pip install langchain langchain-google-genai
```

Step 2 - Imports

Code:

```
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
```

Step 3 - Configure the API key

Code:

```
api_key = "YOUR_GEMINI_API_KEY"
```

Step 4 - Initialize the model

temperature controls how deterministic (0) or creative (1) the output is.

Code:

```
llm = ChatGoogleGenerativeAI(
    model = "gemini-2.5-flash",
    temperature=0.7,
    google_api_key = api_key
)
```

Step 5 - Run a simple prompt

Code:

```
prompt = "Suggest me a skill that is in demand?"
response = llm.invoke(prompt)
print("Suggested Skill:\n", response)
```

Output:

```
Suggested Skill:
AIMessage(content='One in-demand skill right now is Prompt Engineering / applied
AI
development – the ability to design, evaluate and
integrate LLM-powered features into real products.', ...)
```

Step 6 - Create a reusable prompt template

Code:

```
template = "Give me 3 career skills that are in high demand in {year}."
prompt_template = PromptTemplate.from_template(template)
```

Step 7 - Create an output parser

Code:

```
parser = StrOutputParser()
```

Step 8 - Build a chain with LCEL

The pipe operator wires the three pieces together: the template fills in {year}, the LLM generates a response, and the parser returns a clean string.

Code:

```
chain = prompt_template | llm | parser
```

Step 9 - Run the chain

Code:

```
response = chain.invoke({"year": "2026"})
print("\nCareer Skills in 2026:\n", response)
```

Output:

```
Career Skills in 2026:
1. AI/ML Engineering and Applied Generative AI
2. Data Analytics and Data Engineering
3. Cybersecurity and Cloud Infrastructure
```

Applications

- Chatbots and virtual assistants that remember past turns and call external APIs.
- Question answering over PDFs, contracts, and research papers.
- Knowledge management systems that organize and retrieve data from multiple sources.
- Automating multi-step workflows such as customer support or report generation.
- Turning natural-language questions into SQL queries or business insights.

Source: [geeksforgeeks.org/artificial-intelligence/introduction-to-langchain](https://www.geeksforgeeks.org/artificial-intelligence/introduction-to-langchain)

2. Modules of LangChain

LangChain's architecture is deliberately modular: prompt handling, model access, output parsing, retrieval, storage, and agent orchestration are separate, swappable pieces that snap together. The sections below walk through each module with a minimal code example.

1. Prompt Module

Formats raw user input into something an LLM can interpret – either a plain string (PromptTemplate) or a structured, role-based conversation (ChatPromptTemplate).

Code:

```
from langchain_core.prompts import PromptTemplate, ChatPromptTemplate

# String-based prompt
prompt = PromptTemplate.from_template("What is the capital of {country}?")
filled_prompt = prompt.format(country="Germany")
print(filled_prompt)

# Chat-based prompt
chat_prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant."),
    ("human", "{input}")
])
chat_filled = chat_prompt.invoke({"input": "Translate 'hello' to French."})
print(chat_filled.to_messages())
```

Output:

```
What is the capital of Germany?  
[SystemMessage(content='You are a helpful assistant.'),  
 HumanMessage(content="Translate 'hello' to French.")]
```

2. Chat Model / LLM Module

A uniform interface to any supported LLM provider – OpenAI, Anthropic, Gemini, and others.

Code:

```
from langchain_google_genai import ChatGoogleGenerativeAI  
  
llm = ChatGoogleGenerativeAI(model="gemini-2.0-flash", temperature=0)  
response = llm.invoke("What is the capital of Germany?")  
print(response.content)
```

Output:

```
Berlin is the capital of Germany.
```

3. Output Parser Module

Converts a raw LLM string into a structured, predictable format.

Code:

```
from langchain_core.output_parsers import StrOutputParser  
  
parser = StrOutputParser()  
parsed_output = parser.parse(response.content)  
print(parsed_output)
```

Output:

```
Berlin is the capital of Germany.
```

4. Retrieval-Augmented Generation (RAG) Module

Combines document loaders, embeddings, a vector database, and a retriever so the LLM's answer is grounded in real documents rather than only what it memorized during training.

Code:

```
from langchain_core.documents import Document  
from langchain_google_genai import GoogleGenerativeAIEmbeddings  
from langchain.vectorstores import FAISS  
from langchain.chains import RetrievalQA  
  
docs = [  
    Document(page_content="Berlin is the capital of Germany."),  
    Document(page_content="Paris is the capital of France."),  
]  
embeddings = GoogleGenerativeAIEmbeddings(model="models/embedding-001")  
vectorstore = FAISS.from_documents(docs, embeddings)  
retriever = vectorstore.as_retriever()  
qa_chain = RetrievalQA.from_chain_type(llm=llm, retriever=retriever)  
rag_result = qa_chain.run("What is the capital of Germany?")  
print(rag_result)
```

Output:

```
Berlin is the capital of Germany.
```

5. Embedding & Vector Database Module

Embeddings capture the semantic meaning of text as numeric vectors; the vector database stores and indexes those vectors for fast similarity search.

6. Data Storage (HTML, DOC, S3, Web Buckets)

Document loaders bring unstructured content from files, S3 buckets, or web pages into LangChain so it can be chunked, embedded, and stored for retrieval.

Code:

```
from langchain.document_loaders import S3DirectoryLoader

loader = S3DirectoryLoader(bucket="your-bucket", prefix="docs/")
docs = loader.load()
```

7. Agent Module

Agents let an LLM reason over multiple steps and decide which tool to call – a calculator, a web search, a database query – rather than following a fixed script.

Code:

```
from langchain.agents import initialize_agent, Tool

def calc_tool(input_str):
    try:
        return str(eval(input_str))
    except Exception as e:
        return str(e)

tools = [Tool(name="Calculator", func=calc_tool, description="Performs calculations.")]
agent = initialize_agent(tools, llm, agent="zero-shot-react-description", verbose=True)
agent_response = agent.run("What is 5 * 7?")
print(agent_response)
```

Output:

```
> Entering new AgentExecutor chain...
Thought: I should use the Calculator tool to compute 5 * 7.
Action: Calculator
Action Input: 5 * 7
Observation: 35
Thought: I now know the final answer.
Final Answer: 35
```

Source: [geeksforgeeks.org/artificial-intelligence/modules-of-langchain](https://www.geeksforgeeks.org/artificial-intelligence/modules-of-langchain)

3. Environment & API Setup

Before writing LangChain code, it helps to isolate the project in a virtual environment and install the core packages plus whichever model / vector-store integrations you need.

Step 1 - Create and activate a virtual environment

Code:

```
python -m venv .venv

# Windows (PowerShell)
.venv\Scripts\Activate.ps1

# macOS / Linux
source .venv/bin/activate
```

Step 2 - Upgrade pip

Code:

```
pip install --upgrade pip
```

Step 3 - Install core packages

Code:

```
pip install langchain langchain-core langchain-community langchain-openai
langchain-gemini python-dotenv openai
```

Step 4 - Install vector database dependencies

Code:

```
pip install chromadb
pip install faiss-cpu
```

Step 5 - Store API keys in a .env file

Code:

```
OPENAI_API_KEY=your_openai_key_here
GEMINI_API_KEY=your_gemini_key_here
```

Step 6 - Load the keys in code

Code:

```
from dotenv import load_dotenv
import os

load_dotenv()
OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")
GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")
```

Step 7 - Initialize an LLM (OpenAI example)

Code:

```

from langchain.schema import HumanMessage
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv
import os
os.environ["LANGCHAIN_TRACING_V2"] = "false"

load_dotenv()
llm = ChatOpenAI(temperature=0.2, model_name="gpt-3.5-turbo")

response = llm.invoke([HumanMessage(content="Explain LangChain in one
sentence.")])
print("Response:", response.content)

```

Output:

```

Response: LangChain is a framework that helps developers build applications
powered by
    large language models by connecting them to external data, tools, and memory.

```

Step 7b - Initialize an LLM (Gemini example)

Code:

```

from langchain_google_genai import ChatGoogleGenerativeAI
import os
from dotenv import load_dotenv
os.environ["LANGCHAIN_TRACING_V2"] = "false"

load_dotenv()
gemini_api_key = os.getenv("GEMINI_API_KEY")

llm = ChatGoogleGenerativeAI(model="gemini-2.5-pro", api_key=gemini_api_key)
response = llm.invoke("Explain LangChain in one sentence.")
print(response.content)

```

Output:

```

LangChain is a framework that enables developers to build applications powered by
large
    language models by connecting them to external data, tools and memory.

```

Step 8 - Wire up a Chroma vector store

Code:

```

from langchain_community.vectorstores import Chroma
from langchain_openai import OpenAIEmbeddings
from dotenv import load_dotenv
import os

load_dotenv()
OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")

docs = [
    "LangChain connects LLMs with data sources.",
    "Vector databases enable semantic search in RAG systems.",
    "Agents in LangChain can take actions based on tools.",
    "LangChain supports multiple LLM providers like OpenAI, Anthropic, and HuggingFace."
]
embeddings = OpenAIEmbeddings(openai_api_key=OPENAI_API_KEY)
vectordb = Chroma.from_texts(docs, embedding=embeddings,
    persist_directory="./chroma_db")

query = "What can LangChain agents do?"
results = vectordb.similarity_search(query, k=2)
for doc in results:
    print(doc.page_content)

```

Output:

```

Agents in LangChain can take actions based on tools.
LangChain connects LLMs with data sources.

```

Applications

- Chatbots and virtual assistants.
- RAG systems – semantic search over documents using vector stores.
- Agents that perform actions via tools and APIs.
- Content generation – summaries, emails, reports, code.
- Educational tools and tutoring assistants.
- Summarizing and extracting insight from large datasets.

Source: [geeksforgeeks.org/artificial-intelligence/langchain-apis-environment-setup](https://www.geeksforgeeks.org/artificial-intelligence/langchain-apis-environment-setup)

4. Prompt Templates

A prompt template is a reusable, parameterized instruction for an LLM – you define the wording once, with placeholders, and fill in the variable parts at call time. This keeps prompts consistent, reduces repeated boilerplate, and makes prompts easy to test and version.

Types of Prompt Templates

- StringPromptTemplate – simple, text-based templates with placeholders.
- ChatPromptTemplate – multi-role, chat-style templates (covered in the next section).
- FewShotPromptTemplate – embeds example input/output pairs to steer the model's behaviour.

Step 1 - Install requirements

Code:

```
%pip install langchain-community openai
```

Step 2 - Imports

Code:

```
from langchain.prompts import PromptTemplate
from langchain.chat_models import ChatOpenAI
from langchain.chains import LLMChain
from langchain_core.output_parsers import JsonOutputParser
import os
import re
```

Step 3 - Set the API key

Code:

```
os.environ["OPENAI_API_KEY"] = "your_openai_api_key_here"
```

Step 4 - Define a template that asks for structured JSON

Code:

```
prompt = PromptTemplate(
    input_variables=["topic"],
    template="""
Generate a JSON object with the following keys for the topic '{topic}':
- summary: short summary
- key_points: list of 3 key points
- difficulty: "easy", "medium", or "hard"

Output only valid JSON (no explanations outside JSON).
JSON:
"""
)
```

Step 5 - Inspect the filled prompt

Code:

```
filled_prompt = prompt.format(topic="LangChain ChatPromptTemplate")
print(filled_prompt)
```

Output:

```
Generate a JSON object with the following keys for the topic
'LangChain ChatPromptTemplate':
- summary: short summary
- key_points: list of 3 key points
- difficulty: "easy", "medium", or "hard"

Output only valid JSON.
JSON:
```

Step 6 - Initialize the model

Code:

```
llm = ChatOpenAI(model_name="gpt-4", temperature=0)
```

Step 7 - Wrap the prompt + model in a chain**Code:**

```
chain = LLMChain(llm=llm, prompt=prompt)
```

Step 8 - Run the chain**Code:**

```
raw_output = chain.run({"topic": "LangChain ChatPromptTemplate"})  
print(raw_output)
```

Output:

```
{  
  "summary": "ChatPromptTemplate structures multi-turn, role-based prompts for  
chat  
  models.",  
  "key_points": [  
    "Supports system, human, and AI message roles",  
    "Uses variables for dynamic content",  
    "Integrates directly with LLMChain and agents"  
  ],  
  "difficulty": "medium"  
}
```

Step 9 - Extract and parse the JSON**Code:**

```
json_match = re.search(r'\{.*\}', raw_output, re.DOTALL)  
  
if json_match:  
    json_string = json_match.group(0)  
    parser = JsonOutputParser()  
    parsed_output = parser.parse(json_string)  
    print(parsed_output)  
else:  
    print("Could not extract JSON from the output.")
```

Output:

```
{'summary': 'ChatPromptTemplate structures multi-turn, role-based prompts for  
chat  
models.', 'key_points': ['Supports system, human, and AI message roles', 'Uses  
variables  
for dynamic content', 'Integrates directly with LLMChain and agents'],  
'difficulty':  
'medium'}
```

Step 10 - Read individual fields**Code:**

```
print("Summary:", parsed_output.get("summary"))
print("Key Points:")
for point in parsed_output.get("key_points", []):
    print("-", point)
print("Difficulty:", parsed_output.get("difficulty"))
```

Benefits vs. Challenges

- Benefits: reusability, consistency, fewer errors, easier maintenance, better scalability.
- Challenges: variable mismatches, overfitting to few-shot examples, unclear instructions, templates becoming too complex to reason about.

Source: [geeksforgeeks.org/artificial-intelligence/prompt-templates](https://www.geeksforgeeks.org/artificial-intelligence/prompt-templates)

5. ChatPromptTemplate

ChatPromptTemplate is the structured, role-aware counterpart to a plain string template. It's built for multi-turn, chat-style interactions where you need to distinguish system instructions from user and assistant messages.

Core Components

- Roles – who is speaking: system, human, or AI.
- Messages – individual instructions or turns inside the template.
- Variables – placeholders for dynamic content.
- Formatting rules – keep the generated output style consistent.

Step 1 - Install requirements

Code:

```
!pip install langchain openai langchain_community
```

Step 2 - Imports

Code:

```
from langchain.prompts import ChatPromptTemplate, SystemMessagePromptTemplate,
    HumanMessagePromptTemplate
from langchain.chat_models import ChatOpenAI
from langchain.chains import LLMChain
import os
```

Step 3 - Set the API key

Code:

```
os.environ["OPENAI_API_KEY"] = "your-api-key"
```

Step 4 - Define system and human messages

Code:

```
system_message = SystemMessagePromptTemplate.from_template(
    "You are a helpful assistant that provides detailed explanations."
)

human_message = HumanMessagePromptTemplate.from_template(
    "Explain the topic '{topic}' in simple terms suitable for a beginner."
)
```

Step 5 - Combine into a ChatPromptTemplate

Code:

```
chat_prompt = ChatPromptTemplate.from_messages([system_message, human_message])
```

Step 6 - Initialize the model

Code:

```
chat_model = ChatOpenAI(model_name="gpt-4", temperature=0.7)
```

Step 7 - Build the chain

Code:

```
chat_chain = LLMChain(llm=chat_model, prompt=chat_prompt)
```

Step 8 - Run with variables

Code:

```
variables = {"topic": "Machine Learning"}
response = chat_chain.run(variables)
```

Step 9 - Print the response

Code:

```
print("Response:\n", response)
```

Output:

```
Response:
Machine Learning is a way of teaching computers to find patterns in data and make
predictions or decisions without being explicitly programmed for every rule.
Instead of
    writing exact instructions, you show the computer many examples, and it learns
the
    underlying pattern on its own.
```

Applications

- Chatbots that hold structured, context-aware conversations.
- Customer support automation with consistent tone and accuracy.
- Multi-turn Q&A and follow-up handling.
- Interactive, step-by-step tutorials.
- General-purpose conversational agents.

6. Few-Shot Prompting

Few-shot prompting gives the model a handful of input/output examples directly inside the prompt so it can infer the pattern of the task without any fine-tuning. This is called in-context learning – the model's weights don't change, but it conditions its next output on the examples it just read.

How It Works

- 1. User query initialization – the process starts from the task the user wants performed.
- 2. Example source – examples can be static (hand-written) or dynamic (retrieved by semantic similarity from a vector store).
- 3. Retrieval of relevant examples – for dynamic retrieval, the query is embedded and the top-k most similar examples are pulled from the store, sometimes via a full RAG pipeline.
- 4. Prompt construction – the examples, any instructions, and the user query are combined into one structured prompt.
- 5. Model processing – the LLM applies in-context learning, recognizing the pattern in the examples and mapping it onto the new query.
- 6. Output generation – the model produces a response that follows the demonstrated pattern.

Example - Sentiment Classification

Prompt:

```
Classify the sentiment of the following sentences:  
  
Sentence: "I love this product" Sentiment: Positive  
Sentence: "This is the worst experience" Sentiment: Negative  
Sentence: "This app is really easy to use" Sentiment: Positive  
  
Sentence: "The delivery was delayed by a week" Sentiment:
```

Output:

```
Sentiment: Negative
```

Strategies for Good Few-Shot Prompts

- Use clear examples that closely match the target task.
- Include diverse examples to improve generalization.
- Keep the input/output format consistent across examples.
- Prefer a few high-quality examples over many noisy ones.
- Tailor examples to the specific domain or task.
- Test on new, unseen inputs to check the prompt actually generalizes.

Advantages vs. Limitations

- Advantages: minimal training data required, fast to iterate, adapts across many tasks, avoids the cost of fine-tuning.
- Limitations: can be inaccurate on edge cases, may inherit bias from the chosen examples, risk of overfitting to the exact examples shown, weaker on very complex multi-step tasks.

Source: [geeksforgeeks.org/artificial-intelligence/few-shot-prompting](https://www.geeksforgeeks.org/artificial-intelligence/few-shot-prompting)

7. Output Parsers

LLMs return free-form text. Output parsers convert that text into a structured shape – a string, list, JSON object, or a validated Pydantic model – so the rest of your application can consume it reliably instead of scraping the raw string every time.

Types of Output Parsers

- String Parser – plain text passthrough.
- List Parser – splits text into a list/array.
- JSON / Dict Parser – converts output into JSON or a Python dict.
- Pydantic Parser – enforces a strict schema with type validation.
- OutputFixingParser – automatically repairs malformed output to match the expected schema.
- Custom Parser – user-defined logic for specialised formats.

Step 1 – Install requirements

Code:

```
%pip install langchain-community
```

Step 2 – Imports

Code:

```
from langchain.prompts import PromptTemplate
from langchain.chat_models import ChatOpenAI
from langchain.chains import LLMChain
from langchain_core.output_parsers import JsonOutputParser
import json
import os
import re
```

Step 3 – Set the API key

Code:

```
os.environ["OPENAI_API_KEY"] = "your-api-key"
```

Step 4 – Prompt that asks for JSON

Code:

```
prompt = PromptTemplate(
    input_variables=["topic"],
    template="""
Generate a JSON object with the following keys for the topic '{topic}':
- summary: short summary
- key_points: list of 3 key points
- difficulty: "easy", "medium", or "hard"

JSON:
"""
)
```

Step 5 - Initialize the model (deterministic)**Code:**

```
llm = ChatOpenAI(model_name="gpt-4", temperature=0)
```

Step 6 - Chain + Step 7 - Run**Code:**

```
chain = LLMChain(llm=llm, prompt=prompt)
raw_output = chain.run({"topic": "LangChain ChatPromptTemplate"})
```

Step 8 - Extract the JSON substring**Code:**

```
json_match = re.search(r'\{.*\}', raw_output, re.DOTALL)
```

Step 9 - Parse and print**Code:**

```
if json_match:
    json_string = json_match.group(0)
    parser = JsonOutputParser()
    parsed_output = parser.parse(json_string)
    print("Parsed JSON Output:\n", parsed_output)
else:
    print("Could not extract JSON from the output.")
    print("Raw output:\n", raw_output)
```

Output:

```
Parsed JSON Output:
{'summary': 'LangChain ChatPromptTemplate structures multi-turn prompts for chat models.', 'key_points': ['Supports role-based messages', 'Enables dynamic variables', 'Improves consistency across turns'], 'difficulty': 'medium'}
```

Comparison of Parser Types

Parser	Purpose	Strength	Limitation
String	Plain text output	Simple, minimal setup	No structure

List	Splits text into items	Clean multi-item extraction	Needs consistent delimiters
JSON / Dict	Structured data	Direct app integration	Fails on malformed JSON
Pydantic	Schema + type validation	Robust, safe	More setup required
OutputFixing	Repairs bad output	Auto-fixes formatting	Slower, depends on LLM

Source: [geeksforgeeks.org/artificial-intelligence/output-parsers-in-langchain](https://www.geeksforgeeks.org/artificial-intelligence/output-parsers-in-langchain)

8. LLM Chains

An LLM Chain is a structured sequence that combines a model with prompt templates, memory, retrievers, and other tools into one workflow. Chains let you go beyond a single LLM call and build genuinely multi-step applications.

Key Components

- LLM – the generative model at the centre of the chain.
- Prompt templates – format and structure the input.
- Chains – the sequence of steps (single call or multi-step).
- Memory management – retains context across turns.
- Retrievers / vector stores – ground responses in external data (RAG).
- Agents – add dynamic decision-making to the chain.
- Output parsers & postprocessors – clean and format the final result.
- Error handlers – keep the chain robust against bad or malformed output.

Working of an LLM Chain

- User submits a query.
- Preprocessors clean and normalize the input.
- The query is embedded and matched against a vector store, if retrieval is used.
- Retrieved context + input are combined via a prompt template.
- The LLM generates a response from the enriched prompt.
- The raw output is parsed/validated.
- The final response is returned to the user.

A Simple Chain

Code:

```

import textwrap
from langchain_openai import OpenAI
from langchain.chains import LLMChain
from langchain.prompts import PromptTemplate
import os
os.environ["OPENAI_API_KEY"] = "openai_api_key"

llm = OpenAI(temperature=0.7)

prompt = PromptTemplate(
    input_variables=["topic"],
    template="Write a short discription about {topic}."
)

chain = LLMChain(llm=llm, prompt=prompt)

response = chain.run("LLM Chains")
print("\n-----Generated Response-----\n")
print(response)
print("\n-----\n")

```

Output:

```

-----Generated Response-----

LLM Chains are structured pipelines that connect a language model with prompt
templates, memory, and external tools, letting developers build multi-step,
context-aware applications instead of relying on a single, isolated model call.

-----

```

Types of LLM Chains

- Simple sequential chains – one call, or a straight series of calls, output feeds next input.
- Multi-step (complex sequential) chains – break a large task into stages, e.g. chunk → retrieve → summarize.
- Router chains – dynamically pick which sub-chain or model to invoke based on the input.
- Agent-based chains – an agent decides actions and tool calls adaptively.
- RAG chains – ground generation in retrieved documents for factual accuracy.
- Chain-of-Thought / reasoning chains – prompt the model to reason step by step before answering.
- Chain-of-Agents collaboration – multiple specialised agents share outputs to solve long or complex tasks together.

LLM Chains vs. LLM Agents

Aspect	LLM Chains	LLM Agents
Workflow	Static, pre-built steps	Dynamic, chosen at run time
Decision logic	Hard-coded, no deviation	LLM reasons about next action
Tool calls	Fixed sequence	Can choose, skip, or repeat tools
Error handling	Basic fallback steps	Can reason about errors and adapt
Best for	Repeatable pipelines	Open-ended, exploratory problems

9. Agents and Tools

Tools are functions, APIs, or logic that extend what an LLM can do beyond text generation. Agents are LLM-powered planners that reason about a query, pick the right tool(s), observe the results, and adjust – iterating until they can produce a final answer.

Types of Agents

1. OpenAI Function Agent

Uses OpenAI's function-calling API for structured tool invocation – good for form-filling or validated output generation.

Code:

```
from langchain.agents import create_openai_functions_agent, AgentExecutor
from langchain_openai import ChatOpenAI
from langchain_core.tools import tool
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder

@tool
def greet(name: str) -> str:
    """Return a greeting message for the given name."""
    return f"Hello, {name}!"

llm = ChatOpenAI(model="gpt-4-0613")

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant with access to tools."),
    ("human", "{input}"),
    MessagesPlaceholder(variable_name="agent_scratchpad"),
])

agent = create_openai_functions_agent(llm, [greet], prompt=prompt)
agent_executor = AgentExecutor(agent=agent, tools=[greet], verbose=True)

response = agent_executor.invoke({"input": "Greet Alice"})
print(response["output"])
```

Output:

```
Hello, Alice!
```

2. OpenAI Tools Agent

The LLM reasons about which registered tool to call, rather than following a fixed function.

Code:

```

from langchain.agents import create_openai_tools_agent, AgentExecutor
from langchain_openai import ChatOpenAI
from langchain_core.tools import tool
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder

@tool
def add(a: int, b: int) -> int:
    """Add two numbers and return the result."""
    return a + b

llm = ChatOpenAI(model="gpt-3.5-turbo")

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are an assistant that answers using tools."),
    ("human", "{input}"),
    MessagesPlaceholder(variable_name="agent_scratchpad"),
])

agent = create_openai_tools_agent(llm, [add], prompt=prompt)
agent_executor = AgentExecutor(agent=agent, tools=[add], verbose=True)

result = agent_executor.invoke({"input": "What is 5 plus 7?"})
print(result["output"])

```

Output:

```
5 plus 7 is 12.
```

3. ReAct Agent (Reason + Act)

Iterates through explicit Thought → Action → Observation cycles, updating its reasoning as it goes – well suited to multi-step or trial-and-error problems.

Code:

```

from langchain.prompts.chat import ChatPromptTemplate
from langchain.agents import create_react_agent, AgentExecutor
from langchain_openai import ChatOpenAI
from langchain_core.tools import tool

REACT_PROMPT_TEMPLATE = """\
You are a thoughtful agent that reasons and acts in this format:

Thought: you should always think before acting
Action: the action to take, should be one of [{tool_names}]
Action Input: the input to the action
Observation: the result of the action

(Repeat Thought/Action/Action Input/Observation as needed)
Thought: I now know the final answer
Final Answer: the final answer to provide to the user

Tools:
{tools}

Begin!

Question: {input}
{agent_scratchpad}"""

@tool
def echo(text: str) -> str:
    """Return the input text as the output."""
    return text

llm = ChatOpenAI(model="gpt-4")
prompt = ChatPromptTemplate.from_template(REACT_PROMPT_TEMPLATE)
prompt.input_variables.extend(["tools", "tool_names"])

agent = create_react_agent(llm, [echo], prompt=prompt)
agent_executor = AgentExecutor(agent=agent, tools=[echo], verbose=True,
                               handle_parsing_errors=True)

response = agent_executor.invoke({"input": "Echo Hello Geek"})
print(response["output"])

```

Output:

```

Thought: I should use the echo tool.
Action: echo
Action Input: Hello Geek
Observation: Hello Geek
Thought: I now know the final answer.
Final Answer: Hello Geek

```

4. XML Agent

Wraps tool output in XML, useful when enterprise systems expect a strict schema.

Code:

```

@tool
def greet_xml(name: str) -> str:
    """Return a greeting message wrapped in XML tags."""
    return f"<greeting>Hello, {name}</greeting>"

# ... same create_openai_tools_agent / AgentExecutor pattern as above ...
response = agent_executor.invoke({"input": "Greet Bob in XML format."})

```

Output:

```
<greeting>Hello, Bob!</greeting>
```

5. JSON Chat Agent

Handles structured JSON conversations – good for chatbots or API integrations that need schema-based input/output.

Code:

```
def echo_tool(input_text: str) -> str:
    """Echo tool that expects JSON and returns it back with
    'status=processed'."""
    try:
        data = json.loads(input_text)
        data["status"] = "processed"
        return json.dumps(data, indent=2)
    except Exception as e:
        return json.dumps({"error": str(e)})

tools = [Tool(name="EchoJSON", func=echo_tool,
              description="Takes a JSON string and returns it back with
              'status=processed'.")]

llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)
agent = initialize_agent(tools=tools, llm=llm,
                        agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
                        verbose=True)

query = '''Take the following info and return JSON with status:
{"name": "Alice", "age": 25, "city": "New York"}'''

response = agent.run(query)
```

Output:

```
{
  "name": "Alice",
  "age": 25,
  "city": "New York",
  "status": "processed"
}
```

6. Structured Chat Agent

Code:

```

from langchain import hub

@tool
def book_flight(origin: str, destination: str) -> str:
    """Simulate flight booking from origin to destination."""
    return f"Booked flight from {origin} to {destination}"

tools = [book_flight]
prompt = hub.pull("hwchase17/openai-functions-agent")
llm = ChatOpenAI(model="gpt-4", temperature=0)
agent = create_openai_functions_agent(llm=llm, tools=tools, prompt=prompt)
executor = AgentExecutor(agent=agent, tools=tools, verbose=True)

response = executor.invoke({"input": "Book a flight from NYC to LA"})

```

Output:

```
Booked flight from NYC to LA
```

Tools in LangChain

A tool is any function, API, or module an agent can invoke. Tools have a name, a callable, and a description – the agent reads the description to decide when to use it.

Calculator Tool

Code:

```

from langchain.agents import Tool

def calculator(expression: str) -> str:
    try:
        return str(eval(expression))
    except Exception as e:
        return f"Error: {e}"

calc_tool = Tool(name="Calculator", func=calculator,
                 description="Evaluate a math expression like 2+2")

result = calc_tool.run("125 * 12")
print("Calculator Tool Output:", result)

```

Output:

```
Calculator Tool Output: 1500
```

Web Search Tool

Code:

```

from langchain_community.tools import DuckDuckGoSearchRun
from langchain.agents import Tool

search = DuckDuckGoSearchRun()
search_tool = Tool(name="Web Search", func=search.run,
                  description="Search the web and return relevant results")

result = search_tool.run("LangChain Python tutorials")
print("Web Search Tool Output:", result[:100], "...")

```

Output:

```
Web Search Tool Output: In this tutorial, we'll walk through a basic RAG flow
using
Python, LangChain, ChromaDB and OpenAI ...
```

PDF Reader Tool**Code:**

```
from langchain.document_loaders import PyPDFLoader
from langchain.agents import Tool

def read_pdf(file_path: str) -> str:
    loader = PyPDFLoader(file_path)
    docs = loader.load()
    return "\n".join([page.page_content for page in docs])

pdf_tool = Tool(name="PDF Reader", func=read_pdf,
                description="Read a PDF file and return its text content")

result = pdf_tool.run("sample.pdf")
```

Python REPL Tool**Code:**

```
from langchain.agents import Tool

def python_eval(code: str) -> str:
    try:
        return str(eval(code))
    except Exception as e:
        return f"Error: {e}"

python_tool = Tool(name="Python REPL", func=python_eval,
                  description="Run Python code and return the result")

result = python_tool.run("10 + 25")
print("Python REPL Tool Output:", result)
```

Output:

```
Python REPL Tool Output: 35
```

Multi-Tool Agent

An agent isn't limited to one tool – it can hold a whole toolbox and pick whichever tool fits each part of a multi-step request.

Code:

```

from langchain.agents import initialize_agent, AgentType
from langchain.chat_models import ChatOpenAI

tools = [calc_tool, python_tool, pdf_tool, search_tool]
llm = ChatOpenAI(model="gpt-4")

agent = initialize_agent(tools=tools, llm=llm,
                        agent_type=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
                        verbose=True)

response = agent.run(
    "Calculate 15*12, run 10+25, and simulate a web search for LangChain.")
print(response)

```

Output:

```

15 * 12 = 180
10 + 25 = 35
Web search results for 'LangChain' returned several tutorials and the official
documentation.

```

Why Combine Agents and Tools

- Dynamic reasoning – the agent selects tools intelligently.
- Multi-step automation of otherwise manual workflows.
- Easy extensibility – new tools can be added without rewriting the agent.
- Structured interaction – JSON/XML agents enforce a schema.
- Real-time knowledge via API and document access.

Source: [geeksforgeeks.org/artificial-intelligence/agents-and-tools-in-langchain](https://www.geeksforgeeks.org/artificial-intelligence/agents-and-tools-in-langchain)

10. Memory in LangChain

Memory lets a chain or agent carry context from earlier turns into later ones. Without it, every message is handled in isolation and the model can't refer back to anything the user said before.

Types of Memory

- Conversation Buffer Memory – stores the full raw history; simple, but grows unbounded and can blow past token limits in long chats.
- Conversation Buffer Window Memory – keeps only the most recent k exchanges, capping prompt size.
- Conversation Summary Memory – periodically summarizes the conversation with an LLM instead of storing it verbatim, which is far more token-efficient for long chats.
- Entity Memory – extracts and tracks structured facts about specific entities (people, places, topics) mentioned in the conversation.
- Vector-Store Backed Memory – stores snippets in a vector database and retrieves them by similarity, ideal for RAG-style recall.
- Remote Database Memory – persists memory in Redis/DynamoDB for durability and multi-user, multi-session applications.

- LangGraph Memory - a modern persistence layer with branching, orchestration, and deep integration for complex agent applications.

Step 1 - Set the API key

Code:

```
import os
os.environ["OPENAI_API_KEY"] = "your_openai_api_key"
```

Step 2 - Imports

Code:

```
from langchain_openai import ChatOpenAI
from langchain.chains import LLMChain
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder,
    HumanMessagePromptTemplate
from langchain.memory import ConversationBufferMemory
from langchain_core.chat_history import InMemoryChatMessageHistory
```

Step 3 - Initialize chat history

Code:

```
chat_history = InMemoryChatMessageHistory()
```

Step 4 - Configure memory

Code:

```
memory = ConversationBufferMemory(
    memory_key="chat_history",
    chat_memory=chat_history,
    return_messages=True
)
```

Step 5 - Model + prompt with a history placeholder

Code:

```
llm = ChatOpenAI(model_name="gpt-4")

prompt = ChatPromptTemplate.from_messages([
    MessagesPlaceholder(variable_name="chat_history"),
    HumanMessagePromptTemplate.from_template("{query}")
])
```

Step 6 - Chain it together and run

Code:

```
conversation_chain = LLMChain(llm=llm, prompt=prompt, memory=memory,
    verbose=True)

response = conversation_chain.run({"query": "What is AI?"})
print(response)
```

Output:

Artificial Intelligence (AI) is the field of computer science focused on building systems that can perform tasks which normally require human intelligence, such as understanding language, recognizing patterns, and making decisions.

Use Cases

- Conversational agents / chatbots that stay coherent across turns.
- Customer support that recalls previous issues or preferences.
- Long-form content generation that retains context across sections.
- RAG systems that combine retrieval with chat history for accurate, contextual answers.

Limitations

- Memory size constraints – long conversations may need truncation or summarization.
- Context drift – irrelevant accumulated history can reduce answer quality.
- Performance overhead from managing large memory buffers.
- Privacy & security concerns when storing sensitive user data.
- Misconfigured memory or prompts can cause the model to ignore history entirely.

Source: [geeksforgeeks.org/artificial-intelligence/memory-in-langchain-1](https://www.geeksforgeeks.org/artificial-intelligence/memory-in-langchain-1)

11. Vector Stores in LangChain

A vector store is a database specialised for storing embeddings – numeric vectors that capture the meaning of text – and for running fast similarity search over them. Vector stores are the backbone of RAG: embed your documents, store them, retrieve the most relevant chunks for a query, then feed those chunks to the LLM.

Key Terms

- Embedding – a fixed-length numeric vector representing the semantic content of text.
- Vector store / index – stores vectors plus metadata and supports similarity search (k-NN, ANN).
- Retriever – LangChain's abstraction that wraps a vector store and returns the top-k matches for a query.
- ANN vs exact search – exact search checks every vector (accurate but slow at scale); Approximate Nearest Neighbor (ANN) trades a small accuracy loss for much faster lookups.

Step 1 - Install dependencies

Code:

```
!pip install langchain-community langchain chromadb openai
```

Step 2 - Set the API key

Code:

```
import os
os.environ["OPENAI_API_KEY"] = "API_key_here"
```

Step 3 - Imports

Code:

```
from langchain_community.embeddings import OpenAIEmbeddings
from langchain.vectorstores import Chroma
from langchain.docstore.document import Document
```

Step 4 - Create the embedding model

Code:

```
emb = OpenAIEmbeddings(openai_api_key=os.getenv("OPENAI_API_KEY"))
```

Step 5 - Prepare documents

Code:

```
docs = [
    Document(page_content="LangChain connects LLMs with external tools.",
              metadata={"source": "intro"}),
    Document(page_content="Vector stores store embeddings and metadata.",
              metadata={"source": "concepts"})
]
```

Step 6 - Create and persist a Chroma collection

Code:

```
vectordb = Chroma.from_documents(
    documents=docs,
    embedding=emb,
    collection_name="my_collection",
    persist_directory="./chroma_db"
)
vectordb.persist()
```

Step 7 - Create a retriever

Code:

```
retriever = vectordb.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 2}
)
```

Step 8 - Query and print results

Code:

```
results = retriever.get_relevant_documents("What stores vectors?")
for r in results:
    print(r.page_content, r.metadata)
```

Output:

```
Vector stores store embeddings and metadata. {'source': 'concepts'}  
LangChain connects LLMs with external tools. {'source': 'intro'}
```

Choosing a Vector Store

- Small projects / prototyping: Chroma or FAISS – easy setup, runs locally.
- Large-scale (millions of vectors): Pinecone, Milvus, Qdrant, or Weaviate – built for scale.
- Managed vs self-hosted: Pinecone / Chroma Cloud trade cost for reliability; FAISS is free but self-managed.
- Metadata filtering / hybrid search: Weaviate, Qdrant, and Pinecone are strong choices.
- Very large indexes: Milvus is purpose-built for large-scale indexing.
- All six – Chroma, FAISS, Pinecone, Milvus, Qdrant, Weaviate – have first-class LangChain support.

Limitations

- Retrieval quality depends heavily on the embedding model used.
- Changing embedding models means re-embedding and reindexing the whole dataset.
- Large vector datasets carry real storage cost, especially on managed services.
- ANN indexing speeds up queries but can trade off a bit of recall accuracy.

Source: [geeksforgeeks.org/artificial-intelligence/vector-stores-in-langchain](https://www.geeksforgeeks.org/artificial-intelligence/vector-stores-in-langchain)

12. Retrieval-Augmented Generation (RAG) with LangChain

RAG combines a retriever with a generative LLM: instead of relying only on what the model memorized during training, the system fetches relevant external documents at query time and conditions the model's answer on them. This improves factual accuracy, reduces hallucination, and lets you update the knowledge base without retraining the model.

Major Components of a RAG System

- Document loader & chunker – loads raw text/PDF/web content and splits it into manageable pieces.
- Embeddings model – converts chunks and queries into semantic vectors.
- Vector store (index) – stores embeddings and supports similarity search.
- Retriever – finds the top relevant chunks for a given query.
- Generative LLM – produces the final answer from the query plus retrieved context.
- Chain / orchestration – ties the pipeline together end-to-end.
- Memory (optional) – carries conversation history for multi-turn RAG.

End-to-End Implementation (Gemini + FAISS)

Step 1 - Install dependencies

Code:

```
pip install --upgrade langchain langchain-google-genai faiss-cpu
sentence-transformers
google-colab
```

Step 2 - Configure the Gemini API key

Code:

```
import google.generativeai as genai
from google.colab import userdata

GOOGLE_API_KEY = userdata.get('GOOGLE_API_KEY')
genai.configure(api_key=GOOGLE_API_KEY)
```

Step 3 - Load a document

Code:

```
from langchain.docstore.document import Document

doc_text = """
Elon Musk is a technology entrepreneur and engineer known for founding SpaceX and Tesla.
He was born on June 28, 1971, in Pretoria, South Africa.
His major achievements include advancing space exploration and electric vehicles.
Musk is also involved with Neuralink and The Boring Company.
"""

document = Document(page_content=doc_text, metadata={"source": "in-memory-doc"})
```

Step 4 - Split the document into chunks

Code:

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
docs_split = text_splitter.split_documents([document])
```

Step 5 - Local embeddings (no external API call)

Code:

```
from langchain.embeddings import HuggingFaceEmbeddings

embeddings =
HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
```

Step 6 - Build a FAISS vector store

Code:

```
from langchain.vectorstores import FAISS

vectorstore = FAISS.from_documents(docs_split, embeddings)
```

Step 7 - Initialize the Gemini model

Code:

```
from langchain_google_genai import ChatGoogleGenerativeAI

llm = ChatGoogleGenerativeAI(
    model="gemini-2.5-flash",
    temperature=0.0,
    max_tokens=None,
    api_key=GOOGLE_API_KEY,
)
```

Step 8 - Build the RetrievalQA chain

Code:

```
from langchain.chains import RetrievalQA

retriever = vectorstore.as_retriever()

qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=retriever,
    return_source_documents=True,
    chain_type="stuff",
)
```

Step 9 - Query and print the grounded answer

Code:

```
query = "Who is Elon Musk and what are his major achievements?"

result = qa_chain.invoke({"query": query})

print("Answer:", result['result'])
print("\nSource Documents:")
for doc in result['source_documents']:
    print(f"- {doc.page_content}")
```

Output:

```
Answer: Elon Musk is a technology entrepreneur and engineer known for founding
SpaceX
    and Tesla. Born on June 28, 1971, in Pretoria, South Africa, his major
achievements
    include advancing space exploration and electric vehicles. He is also involved
with
    Neuralink and The Boring Company.

Source Documents:
- Elon Musk is a technology entrepreneur and engineer known for founding SpaceX
and
    Tesla. He was born on June 28, 1971, in Pretoria, South Africa...
```

Further Enhancements

- Add memory (e.g. ConversationBufferMemory) for multi-turn RAG.
- Customize prompt templates for tighter control over the answer format.

- Swap in Pinecone, Weaviate, or Chroma for scalability or managed hosting.
- Use OpenAI, Vertex AI, or other transformer embeddings instead of local models.
- Load documents dynamically from PDFs, websites, or databases.
- Use 'map-reduce' or 'refine' chain types for very large document collections.

Source: [geeksforgeeks.org/artificial-intelligence/rag-with-langchain](https://www.geeksforgeeks.org/artificial-intelligence/rag-with-langchain)

13. LangChain Expression Language (LCEL)

LCEL connects prompts, models, retrievers, and parsers with the pipe operator (`|`), so a chain reads left-to-right as data flowing from one block to the next. It replaces a lot of boilerplate chain-building code with a compact, declarative expression.

Key Features

- Declarative syntax – chains are built by piping `Runnable`s together.
- Parallel execution – independent branches can run concurrently via `RunnableParallel`.
- Async by default – every chain supports asynchronous execution out of the box.
- Streaming – supports incremental token streaming for faster perceived response time.
- Deployment – chains can be served directly via `LangServe` with retries and scaling.

Runnables Interface

At the core of LCEL are `Runnables` – modular units that wrap a function or operation and can be chained. Any two `Runnables` can be combined with the pipe operator, and the output of one `.invoke()` becomes the input to the next.

A Simple LCEL Chain (Cohere)

Code:

```

from langchain_core.prompts import PromptTemplate
from langchain_cohere import ChatCohere
from langchain.schema.output_parser import StrOutputParser
import os

os.environ["cohere_api_key"] = "your_key_here"

llm = ChatCohere(model="command-r", temperature=0,
                 cohere_api_key=os.environ["cohere_api_key"])

template = """Question: {question}

Answer: Let's think step by step."""
prompt = PromptTemplate.from_template(template)
output_parser = StrOutputParser()

# Compose with the pipe operator
chain = prompt | llm | output_parser

question = "I have five apples. I throw two away. I eat one. How many apples do I have left?"
response = chain.invoke({"question": question})
print(response)

```

Output:

```

You started with five apples, removed two by throwing them away, and then
consumed one
    more, which leaves you with two apples.

So, the final answer is you have two apples left.

```

Types of Runnables

RunnablePassthrough

Code:

```

from langchain_core.runnables import RunnablePassthrough

passthrough = RunnablePassthrough()
result = passthrough.invoke("Hello, Geek!")
print(result)

```

Output:

```

Hello, Geek!

```

RunnableParallel

Code:

```

from langchain_core.runnables import RunnableParallel, RunnableLambda

def to_uppercase(x): return x.upper()
def word_count(x): return len(x.split())

uppercase = RunnableLambda(to_uppercase)
count_words = RunnableLambda(word_count)

parallel = RunnableParallel({"upper": uppercase, "count": count_words})
result = parallel.invoke("LangChain makes AI development easier")
print(result)

```

Output:

```
{'upper': 'LANGCHAIN MAKES AI DEVELOPMENT EASIER', 'count': 5}
```

RunnableLambda

Code:

```

from langchain_core.runnables import RunnableLambda

def add_five(x):
    return x + 5

def multiply_by_two(x):
    return x * 2

add_five = RunnableLambda(add_five)
multiply_by_two = RunnableLambda(multiply_by_two)

chain = add_five | multiply_by_two
chain.invoke(3)

```

Output:

```
16
```

invoke / batch / stream

Code:

```

prompt_str = "You know 1 short line about {topic}?"
prompt = ChatPromptTemplate.from_template(prompt_str)
chain = prompt | llm | output_parser

result_with_invoke = chain.invoke("AI")

result_with_batch = chain.batch(["AI", "LLM", "Vector Database"])
print(result_with_batch)

for chunk in chain.stream("Artificial Intelligence write 5 lines"):
    print(chunk, flush=True, end="")

```

Output:

```
['AI simulates human intelligence in machines.',  
'An LLM predicts and generates human-like text from patterns in language.',  
'A Vector Database stores and searches data by semantic similarity.']  
Artificial Intelligence lets machines learn from data,  
recognize patterns, make decisions, adapt over time,  
and automate tasks once thought to need human judgment.
```

Advantages vs. Limitations

- Advantages: less boilerplate, faster iteration, built-in parallelism and streaming, automatic LangSmith tracing for debugging, works across RAG, chat, and automation use cases.
- Limitations: fundamentally linear, so complex branching/state logic is awkward; coordinating many external tools inside one LCEL chain isn't very intuitive; long, nested chains can be hard to debug.

Source: [geeksforgeeks.org/artificial-intelligence/langchain](https://www.geeksforgeeks.org/artificial-intelligence/langchain)

14. Further Topics & Reference Links

The GeeksforGeeks LangChain tutorial also links out to a wide set of adjacent articles – ecosystem tools, integrations, and full project walkthroughs. These build directly on the concepts already covered above (chains, agents, memory, vector stores, RAG, LCEL), so rather than duplicate that material, they're listed here as a roadmap for further reading once you're comfortable with the core building blocks.

Ecosystem Tools

- LangGraph – a graph-based orchestration layer for building stateful, multi-actor agent applications with branching and cycles (something linear LCEL chains can't easily express).
- LangChain vs. LangGraph – when to reach for a simple chain vs. a full agent graph.
- LangSmith – observability and tracing for LangChain / LangGraph apps: inspect every prompt, tool call, and intermediate step.
- Debugging & Testing LLMs in LangSmith – evaluation datasets, run comparisons, and regression testing for LLM pipelines.

Integrations

- LangChain + LlamaIndex – combining LangChain's orchestration with LlamaIndex's indexing strengths.
- Hugging Face with LangChain – using open-source models and pipelines as drop-in LLMs.
- Knowledge Graphs – grounding RAG in graph-structured relationships rather than flat text chunks.

Practical, End-to-End Projects

- Build a RAG pipeline with fully open-source models.

- A chatbot web app built with LangChain.
- A PDF summarizer LLM application.
- RAG using Llama 3.
- Preparing documents for Gen-AI workflows (Docling).

Indexing

Indexing keeps a vector store in sync with an evolving source of documents – detecting new, changed, or deleted content and avoiding redundant re-embedding, which matters once a RAG system moves from a demo into production.

Streaming, Tracing & Callbacks

- Streaming Responses – return tokens to the user as they're generated instead of waiting for the full completion (already touched on via LCEL's `.stream()` above).
- Tracing with LangChain – step-by-step visibility into what a chain or agent did, typically via LangSmith.
- LangChain Callbacks – hooks that fire on events (LLM start/end, tool call, error) for logging, monitoring, or custom side effects.

For the live, most up-to-date version of any of these topics, see the original tutorial index at [geeksforgeeks.org/data-science/langchain-tutorial](https://www.geeksforgeeks.org/data-science/langchain-tutorial).